# Ultrasonic Sonar Sensor (HC- SR04)

The ultrasonic sonar sensor is a device designed to measure distance using emission and reception of high-frequency sound waves. Due to this simple working method, it is used in many projects such as obstacle detection, automatic lights, etc. It is reliable, low-cost and easy to connect.

It functions by measuring the time it takes for a 40 kHz pulse to bounce off an object and return to the receiver. The distance is calculated using the speed of sound with the formula:

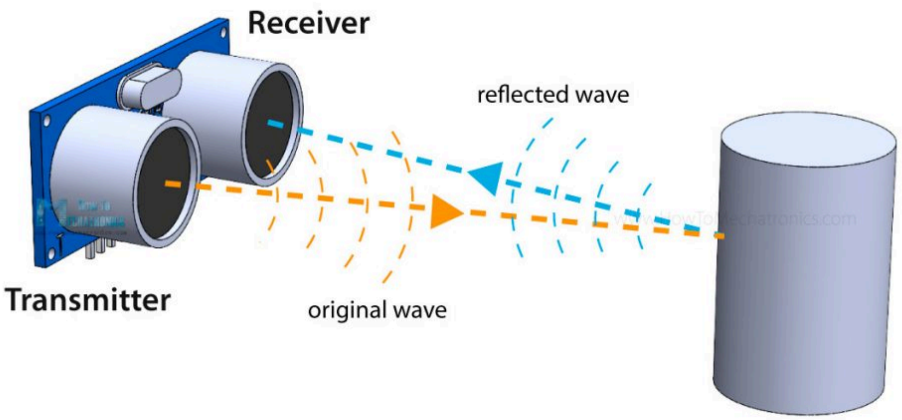
Distance = (Pulse duration x Speed of sound) / 2

Or,

Distance (cm) = Pulse Duration (µs) / 58

Distance (inch) = Pulse Duration (µs) / 148

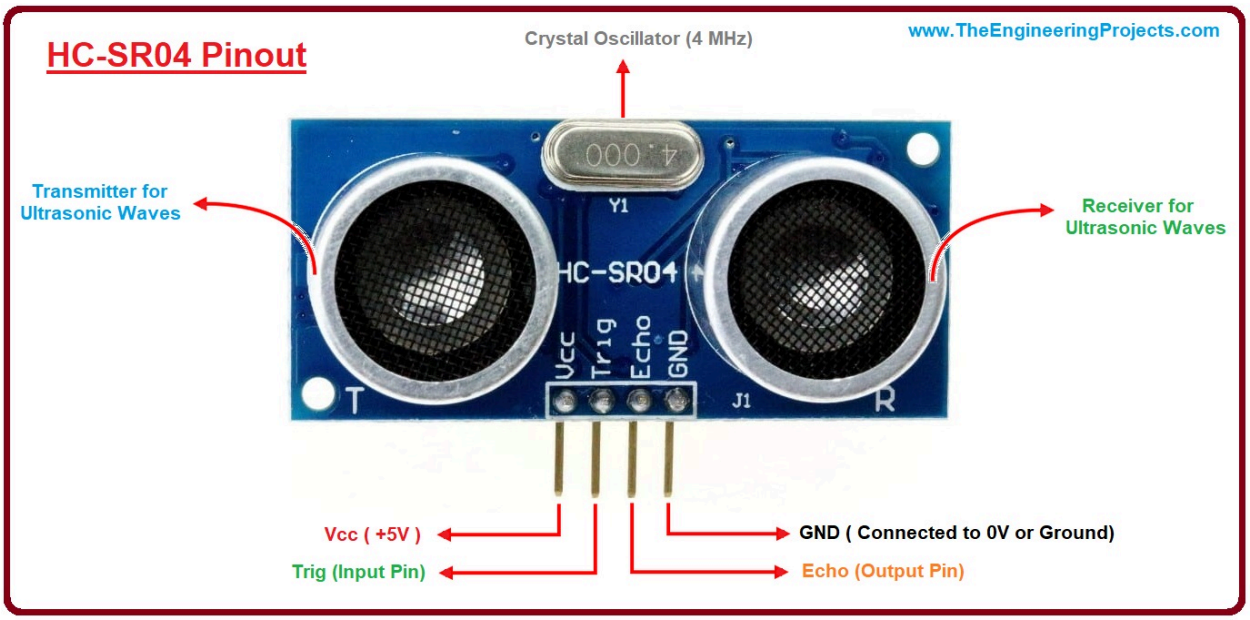
To ensure accurate readings, we should wait at least 60 ms before starting the next measurement.



## Pin Diagram

The sensor has 4 pins:

| Pin Name | Direction | Description |
|---|---|---|
| Vcc | Input | Powers the sensor |
| Trig | Input | Receives trigger pulse from Arduino |
| Echo | Output | Sends pulse duration back to Arduino |
| GND | Input | Ground reference |

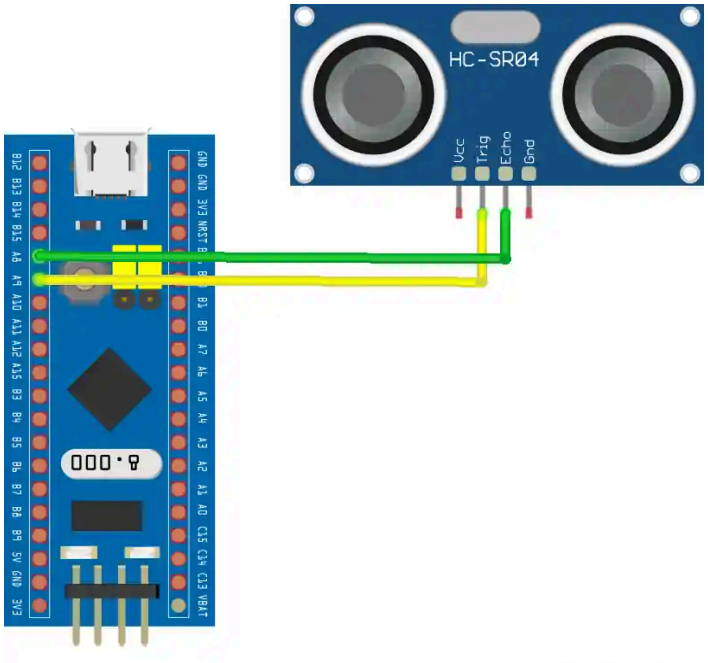


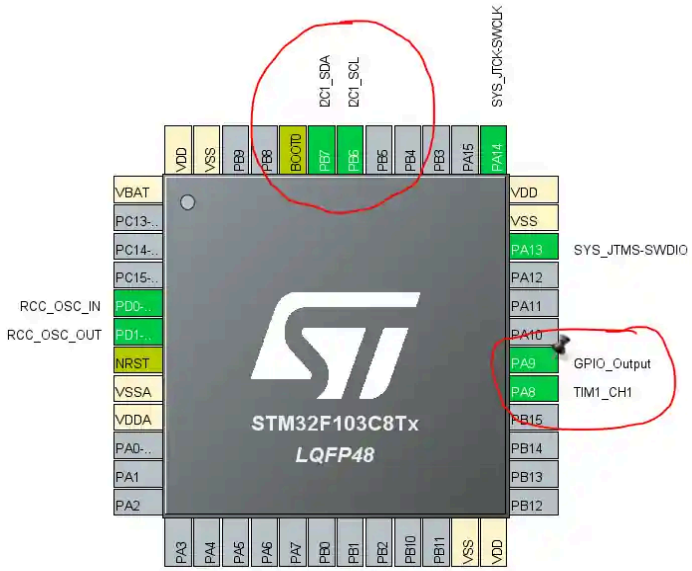

Requirements:

1. Voltage: 5V
2. Trigger Pulse Width: Minimum 10 µs
3. Echo Output Voltage: 5V
4. Measuring Range: 2cm to 400cm
5. Accuracy: Around ±3 mm
6. Ultrasonic Frequency: 40 kHz

# Connecting HC-SR04 with STM32

- Connect the sensor's Vcc to the STM32's 5V.
- Connect the sensor’s GND to the STM32’s GND.
- The trig pin from the sensor is connected to the pin PA9
- The echo pin from the sensor is connected to PA8, the TIM1 CH1 pin.



# Sample C code

```
#include "stm32f4xx_hal.h"

/* USER VARIABLES */

TIM_HandleTypeDef htim1;

#define TRIG_PIN GPIO_PIN_9
#define TRIG_PORT GPIOA

#define ECHO_PIN GPIO_PIN_8
#define ECHO_PORT GPIOA

uint32_t IC_Val1 = 0;
uint32_t IC_Val2 = 0;
uint32_t Difference = 0;

uint8_t Is_First_Captured = 0;
float Distance = 0;


/* FUNCTION PROTOTYPES */

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM1_Init(void);
void delay_us(uint16_t us);
void HCSR04_Read(void);


/* MICROSECOND DELAY USING TIMER */

void delay_us(uint16_t us)
{
	__HAL_TIM_SET_COUNTER(&htim1,0);
	while
(__HAL_TIM_GET_COUNTER(&htim1) < us);
}


/* TRIGGER SENSOR */

void HCSR04_Read (void)
{
	HAL_GPIO_WritePin(TRIG_PORT,
TRIG_PIN, GPIO_PIN_SET);
	delay_us(10);  // 10 microseconds
	HAL_GPIO_WritePin(TRIG_PORT,
TRIG_PIN, GPIO_PIN_RESET);

	__HAL_TIM_ENABLE_IT(&htim1,
TIM_IT_CC1);
}


/* INPUT CAPTURE CALLBACK */

void
HAL_TIM_IC_CaptureCallback(TIM_HandleTypeD
ef *htim)
{
	if (htim->Channel ==
HAL_TIM_ACTIVE_CHANNEL_1)
	{
		if (Is_First_Captured==0)
		{
			IC_Val1 =
HAL_TIM_ReadCapturedValue(htim,
TIM_CHANNEL_1);
			Is_First_Captured = 1;


__HAL_TIM_SET_CAPTUREPOLARITY(htim,
			TIM_CHANNEL_1,

TIM_INPUTCHANNELPOLARITY_FALLING);
		}

		else if (Is_First_Captured==1)
		{
			IC_Val2 =
HAL_TIM_ReadCapturedValue(htim,
TIM_CHANNEL_1);


__HAL_TIM_SET_COUNTER(htim, 0);

			if (IC_Val2 > IC_Val1)
			{
				Difference =
IC_Val2 - IC_Val1;
			}

			else if (IC_Val1 >
IC_Val2)
			{
				Difference =
(0xffff - IC_Val1) + IC_Val2;
			}

			Distance = Difference *
0.034 / 2;

			Is_First_Captured = 0;


__HAL_TIM_SET_CAPTUREPOLARITY(htim,
			TIM_CHANNEL_1,

TIM_INPUTCHANNELPOLARITY_RISING);


__HAL_TIM_DISABLE_IT(&htim1,
TIM_IT_CC1);
```

```
		}
	}
}


/* MAIN FUNCTION */

int main(void)
{
	HAL_Init();
	SystemClock_Config();

	MX_GPIO_Init();
	MX_TIM1_Init();

	HAL_TIM_IC_Start_IT(&htim1, TIM_CHANNEL_1);
	HAL_TIM_Base_Start(&htim1);

	while (1)
	{
		HCSR04_Read();

		HAL_Delay(100);

		// Distance value available in variable "Distance"
	}
}


/* TIMER1 INIT */

static void MX_TIM1_Init(void)
{
	TIM_IC_InitTypeDef sConfigIC = {0};

	htim1.Instance = TIM1;
	htim1.Init.Prescaler = 84-1;
	htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
	htim1.Init.Period = 65535;
	htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
	HAL_TIM_IC_Init(&htim1);

	sConfigIC.ICPolarity = TIM_INPUTCHANNELPOLARITY_RISING;
	sConfigIC.ICSelection = TIM_ICSELECTION_DIRECTTI;
	sConfigIC.ICPrescaler = TIM_ICPSC_DIV1;
	sConfigIC.ICFilter = 0;

	HAL_TIM_IC_ConfigChannel(&htim1, &sConfigIC, TIM_CHANNEL_1);
}


/* GPIO INIT */

static void MX_GPIO_Init(void)
{
	GPIO_InitTypeDef GPIO_InitStruct = {0};

	__HAL_RCC_GPIOA_CLK_ENABLE();

	/* TRIG PIN OUTPUT */

	GPIO_InitStruct.Pin = TRIG_PIN;
	GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
	GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

	HAL_GPIO_Init(TRIG_PORT, &GPIO_InitStruct);

	/* ECHO PIN INPUT CAPTURE */

	GPIO_InitStruct.Pin = ECHO_PIN;
	GPIO_InitStruct.Mode = GPIO_MODE_AF_PP;
	GPIO_InitStruct.Pull = GPIO_NOPULL;
	GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
	GPIO_InitStruct.Alternate = GPIO_AF1_TIM1;

	HAL_GPIO_Init(ECHO_PORT, &GPIO_InitStruct);
}
```

References:


1. HC-SR04 Ultrasonic Sensor with STM32 | Distance Measurement
2. Ultrasonic Sensor Project: Distance Measurement with STM32 | STM32 …
3. Interface HC-SR04 Ultrasonic Sensor with STM32 | Input Capture Tutorial